

SE3082 - Parallel Computing [2026/JUL]

LAB 04

BSc (Hons) in Computer Science

Year 3 Semester 1

IT24103462 – Samarakoon S.M.T.C

Exercise 01

```
Complete!
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_serial.c -o lab04_serial.o
[ec2-user@ip-10-0-0-183 ~]$ ./lab04_serial.o
Pi = 3.141592653590426
Time = 0.326636 seconds
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_race.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_race.c -o lab04_race.o
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
./lab04_race.o
Pi = 0.920418190846552
Time = 0.325279 seconds
[ec2-user@ip-10-0-0-183 ~]$ nano export OMP_NUM_THREADS=4
./lab04_race.o


Use "fg" to return to nano.

[1]+  Stopped                 nano export OMP_NUM_THREADS=4
Pi = 1.106462403695705
Time = 0.326045 seconds
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_critical.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_critical.c -o lab04_critical.o
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=1
./lab04_critical.o
Pi = 3.141592653590426
Time = 2.003206 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=2
./lab04_critical.o
Pi = 3.141592653589928
Time = 2.067605 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
./lab04_critical.o
Pi = 3.141592653590219
Time = 1.690281 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=8
./lab04_critical.o
Pi = 3.141592653589891
Time = 1.799048 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=16
./lab04_critical.o
Pi = 3.141592653589329
Time = 2.040932 seconds
```

```
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp atomic.c -o atomic.o
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=1
[ec2-user@ip-10-0-0-183 ~]$ ./atomic.o
Pi = 3.141592653590426
Time = 1.585144 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=2
[ec2-user@ip-10-0-0-183 ~]$ ./atomic.o
Pi = 3.141592653589855
Time = 2.372242 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
[ec2-user@ip-10-0-0-183 ~]$ ./atomic.o
Pi = 3.141592653590255
Time = 1.595947 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=8
[ec2-user@ip-10-0-0-183 ~]$ ./atomic.o
Pi = 3.141592653589766
Time = 1.561460 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=16
[ec2-user@ip-10-0-0-183 ~]$ ./atomic.o
Pi = 3.141592653589332
Time = 1.587646 seconds
```

02).The answer is correct after applying synchronization. Both the critical and atomic implementations produced π values that were extremely close to the expected value of 3.141592653589793. This confirms that the data race was successfully addressed.

03). Lab 03,

Program	Threads	Pi	Time (s)
Data Race	4	0.920418190846552	0.325279
Data Race	4	1.106462403695705	0.326045

Synchronized versions,

Threads	Critical Time (s)	Atomic Time (s)
1	2.003206	1.585144
2	2.067605	2.372242
4	1.690281	1.595947

8	1.799048	1.561460
16	2.040932	1.587646

04).The serial implementation was considerably faster than the synchronized parallel implementations in this experiment. This is because the critical and atomic versions perform synchronization on the shared accumulation, which introduces additional overhead. Therefore, adding threads did not automatically improve performance.

05).

```
[ec2-user@ip-10-0-0-183 ~]$ nproc
16
[ec2-user@ip-10-0-0-183 ~]$ lscpu | grep -E 'CPU\(s\)|Core|Thread'
CPU(s):                  1
On-line CPU(s) list:     0
Thread(s) per core:      1
Core(s) per socket:      1
NUMA node0 CPU(s):       0
[ec2-user@ip-10-0-0-183 ~]$ lscpu
Architecture:            x86_64
  CPU op-mode(s):        32-bit, 64-bit
  Address sizes:         46 bits physical, 48 bits virtual
  Byte Order:            Little Endian
CPU(s):                  1
  On-line CPU(s) list:   0
Vendor ID:               GenuineIntel
  Model name:            Intel(R) Xeon(R) CPU E5-2686 v4 @ 2.30GHz
    CPU family:          6
    Model:               79
    Thread(s) per core:  1
    Core(s) per socket:  1
    Socket(s):           1
    Stepping:            1
    BogoMIPS:            4599.99
    Flags:               fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr ss
                         e sse2 ht syscall nx rdtscp lm rep_good nopl xtopology cpuid tsc_known_freq pni pclmulqdq s
                         sse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c
                         rdrand hypervisor lahf_lm abm pti fsgsbase bmi1 avx2 smep bmi2 erms invpcid xsaveopt
Virtualization features:
  Hypervisor vendor:     Xen
  Virtualization type:   full
Caches (sum of all):
  L1d:                   32 KiB (1 instance)
```

```
    Flags:               fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr ss
                         e sse2 ht syscall nx rdtscp lm rep_good nopl xtopology cpuid tsc_known_freq pni pclmulqdq s
                         sse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c
                         rdrand hypervisor lahf_lm abm pti fsgsbase bmi1 avx2 smep bmi2 erms invpcid xsaveopt
Virtualization features:
  Hypervisor vendor:     Xen
  Virtualization type:   full
Caches (sum of all):
  L1d:                   32 KiB (1 instance)
  L1i:                   32 KiB (1 instance)
  L2:                    256 KiB (1 instance)
  L3:                    45 MiB (1 instance)
NUMA:
  NUMA node(s):          1
  NUMA node0 CPU(s):     0
Vulnerabilities:
  Gather data sampling:  Not affected
  Ghostwrite:            Not affected
  Indirect target selection: Mitigation; Aligned branch/return thunks
  Itlb multihit:         KVM: Mitigation: VMX unsupported
  L1tf:                  Mitigation; PTE Inversion
  Mds:                   Vulnerable: Clear CPU buffers attempted, no microcode; SMT Host state unknown
  Meltdown:              Mitigation; PTI
  Mmio stale data:       Vulnerable: Clear CPU buffers attempted, no microcode; SMT Host state unknown
  Old microcode:         Not affected
  Reg file data sampling: Not affected
  Retbleed:              Not affected
  Spec rstack overflow:  Not affected
  Spec store bypass:     Vulnerable
  Spectre v1:            Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:            Mitigation; Retpolines; STIBP disabled; RSB filling; PBRSB-eIBRS Not affected; BHI Retpolin
                         e
  Srbds:                 Not affected
  Tsa:                   Not affected
  Tsx async abort:       Not affected
  Vmscape:               Not affected
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex2_serial.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex2_serial.c -o lab04_ex2_serial.o -lm
[ec2-user@ip-10-0-0-183 ~]$ ./lab04_ex2_serial.o
Sum = -3333333333324253448110080.000000000000000
Time = 0.363017 seconds
[ec2-user@ip-10-0-0-183 ~]$ lab04_ex2_race.c
-bash: lab04_ex2_race.c: command not found
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex2_race.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex2_race.c -o lab04_ex2_race.o -lm
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=1
./lab04_ex2_race.o
Sum = -3333333333324253448110080.000000000000000
Time = 0.364809 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=2
./lab04_ex2_race.o
Sum = -3017156484868994993408000.000000000000000
Time = 0.366960 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
./lab04_ex2_race.o
Sum = -1618576771188795386101760.000000000000000
Time = 0.362889 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=8
./lab04_ex2_race.o
```

The lscpu command reported 1 CPU/logical processor exposed to the EC2 instance, with 1 core per socket and 1 thread per core. Although nproc returned 16, this should not be interpreted as 16 physical cores because it is inconsistent with the detailed lscpu information.

06). A higher EC2 configuration could not be created during the experiment due to limitations in the available AWS resources/account. Therefore, the serial and parallel programs could not be tested on a second EC2 configuration. The performance comparison in this experiment is consequently based on the available EC2 instance.

07).

Critical

Threads	Time (s)	Speedup
1	2.003206	1.0000
2	2.067605	0.9689
4	1.690281	1.1851
8	1.799048	1.1135
16	2.040932	0.9815

Atomic

Threads	Time (s)	Speedup
1	1.585144	1.0000
2	2.372242	0.6679
4	1.595947	0.9932
8	1.561460	1.0152
16	1.587646	0.9984

The results show that increasing the number of threads did not consistently improve performance on the available EC2 instance. The critical implementation achieved its lowest execution time with 4 threads, while the atomic implementation achieved its lowest execution time with 8 threads. However, the speedups were relatively small because synchronization of the shared sum introduces overhead. The higher EC2 configuration could not be tested due to AWS resource limitations, so a comparison between two EC2 configurations was not possible.

Exercise 02

Serial program

Program	Time (s)	Sum
Serial	**0.363017**	-33333333332425344811008

Parallel program

Threads	Race Time (s)	Critical Time (s)	Atomic Time (s)
1	0.364809	1.741680	1.559831
2	0.366960	1.758083	1.594425
4	0.362889	1.909066	1.594820
8	0.364469	2.050123	1.568540
16	0.366008	1.864013	1.600494

The race version produce different results ,

Threads	Race Result
1	-333333333332425344811008
2	-3017156484868949934080000
4	-161857677118879538610176
8	-78019250283601337516032
16	-78604091291821766868992

```
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex2_serial.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex2_serial.c -o lab04_ex2_serial.o -lm
[ec2-user@ip-10-0-0-183 ~]$ ./lab04_ex2_serial.o
Sum = -3333333333324253448110080.000000000000000
Time = 0.363017 seconds
[ec2-user@ip-10-0-0-183 ~]$ lab04_ex2_race.c
-bash: lab04_ex2_race.c: command not found
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex2_race.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex2_race.c -o lab04_ex2_race.o -lm
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=1
./lab04_ex2_race.o
Sum = -3333333333324253448110080.000000000000000
Time = 0.364809 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=2
./lab04_ex2_race.o
Sum = -3017156484868949934080000.000000000000000
Time = 0.366960 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
./lab04_ex2_race.o
Sum = -1618576771188795386101760.000000000000000
Time = 0.362889 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=8
./lab04_ex2_race.o
Sum = -7801925028360133751603200.000000000000000
Time = 0.364469 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=16
./lab04_ex2_race.o
Sum = -7860409129182176686899200.000000000000000
Time = 0.366008 seconds
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex2_critical.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex2_critical.c -o lab04_ex2_critical.o -lm
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=1
./lab04_ex2_critical.o
Sum = -3333333333324253448110080.000000000000000
Time = 1.741680 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=2
./lab04_ex2_critical.o
Sum = -3333333333324174930739200.000000000000000
Time = 1.758083 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
./lab04_ex2_critical.o
Sum = -3333333333324337334190080.000000000000000
Time = 1.909066 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=8
./lab04_ex2_critical.o
Sum = -3333333333323526659112960.000000000000000
Time = 2.050123 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=16
./lab04_ex2_critical.o
Sum = -3333333333323773619732480.000000000000000
```

```
./lab04_ex2_critical.o
Sum = -3333333333323773619732480.000000000000000
Time = 1.864013 seconds
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex2_atomic.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex2_atomic.c -o lab04_ex2_atomic.o -lm
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=1
./lab04_ex2_atomic.o
Sum = -3333333333324253448110080.000000000000000
Time = 1.559831 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=2
./lab04_ex2_atomic.o
Sum = -3333333333324219893678080.000000000000000
Time = 1.594425 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
./lab04_ex2_atomic.o
Sum = -3333333333324358809026560.000000000000000
Time = 1.594820 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=8
./lab04_ex2_atomic.o
Sum = -3333333333323742078566400.000000000000000
Time = 1.568540 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=16
./lab04_ex2_atomic.o
Sum = -3333333333323870927585280.000000000000000
Time = 1.600494 seconds
```

Exercise 03

Serial result

Program	Points Inside	Pi	Time (s)
Serial	7,853,466	3.1413864	**0.175974**

Parallel results

Data-race version

Threads	Points Inside	Pi	Time (s)
1	7,853,466	3.1413864	0.174426
2	7,853,163	3.1412652	0.174936
4	7,852,191	3.1408764	0.174033
8	—	3.1411240	0.215317*
16	7,853,850	3.1415400	0.176512

Critical/mutex version

Threads	Points Inside	Pi	Time (s)
1	7,853,466	3.1413864	0.215376
2	7,853,163	3.1412652	0.213999
4	7,852,191	3.1408764	0.215544
8	7,852,810	3.1411240	0.215317
8 (repeat)	7,852,810	3.1411240	0.216717

So the synchronized parallel version was actually slower than the serial version on EC2 instance.

```
[ec2-user@ip-10-0-0-183 ~]$ ./lab04_ex3_serial.o
Pi = 3.141386400000000
Points inside = 7853466
Time = 0.175974 seconds
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex3_race.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex3_race.c -o lab04_ex3_race.o
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=1
./lab04_ex3_race.o
Pi = 3.141386400000000
Points inside = 7853466
Time = 0.174426 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=2
./lab04_ex3_race.o
Pi = 3.141265200000000
Points inside = 7853163
Time = 0.174936 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
./lab04_ex3_race.o
Pi = 3.140876400000000
Points inside = 7852191
Time = 0.174033 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
./lab04_ex3_race.o
Pi = 3.140876400000000
Points inside = 7852191
Time = 0.174580 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=16
./lab04_ex3_race.o
Pi = 3.141540000000000
Points inside = 7853850
Time = 0.176512 seconds
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex3_critical.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex3_critical.c -o lab04_ex3_critical.o
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=1
./lab04_ex3_critical.o
Pi = 3.141386400000000
Points inside = 7853466
Time = 0.215376 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=2
./lab04_ex3_critical.o
Pi = 3.141265200000000
Points inside = 7853163
Time = 0.213999 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
./lab04_ex3_critical.o
Pi = 3.140876400000000
Points inside = 7852191
Time = 0.215544 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=8
./lab04_ex3_critical.o
Pi = 3.141124000000000
Points inside = 7852810
Time = 0.215317 seconds
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=8
./lab04_ex3_critical.o
Pi = 3.141124000000000
Points inside = 7852810
Time = 0.216717 seconds
```

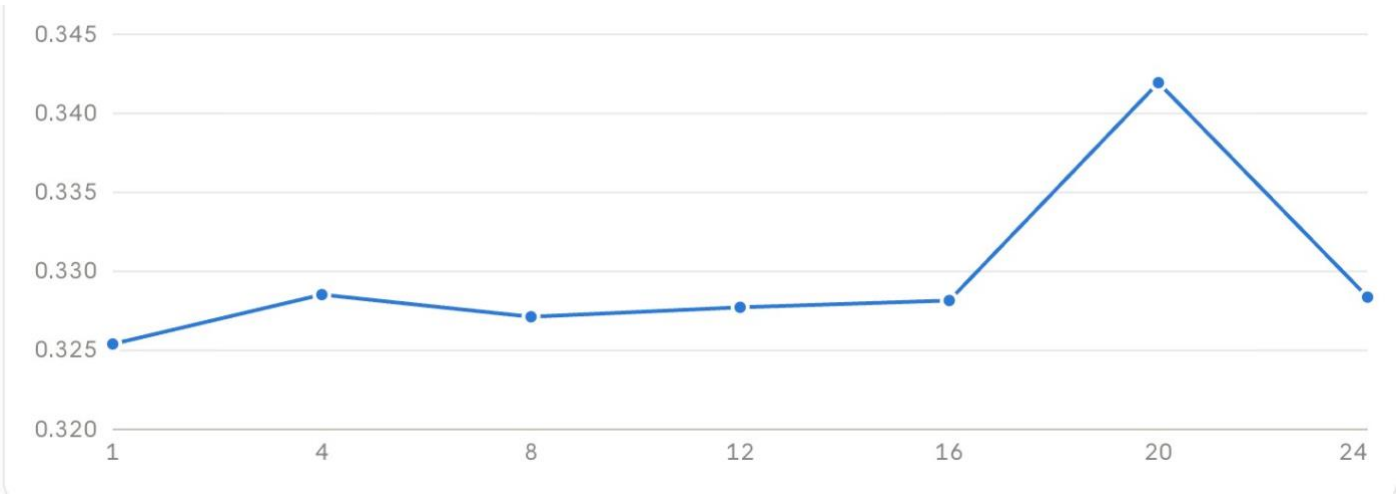
Exercise 04

Average timing

Threads	Average Time (s)
1	0.325405
4	0.328530
8	0.327129
12	0.327726
16	0.328157
20	0.341943
24	0.328371

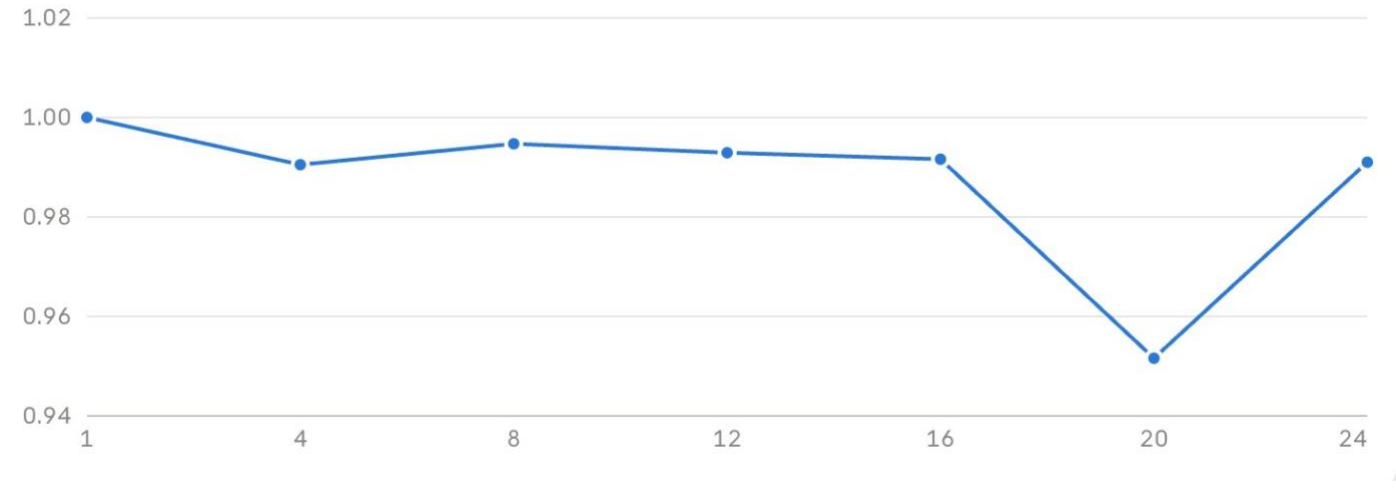
Threads vs Time graph

Threads	Time (s)
1	0.325405
4	0.328530
8	0.327129
12	0.327726
16	0.328157
20	0.341943
24	0.328371



Speedup

Threads	Time (s)	Speedup
1	0.325405	1.0000
4	0.328530	0.9905
8	0.327129	0.9947
12	0.327726	0.9929
16	0.328157	0.9916
20	0.341943	0.9516
24	0.328371	0.9910



Scheduling with 24 threads

Scheduling Method	Threads	Time (s)
Static	24	0.327397
Dynamic	24	2.679877
Guided	24	0.325920

The dynamic schedule was significantly slower than static and guided scheduling. This is because dynamic scheduling introduces additional runtime scheduling overhead. Since the iterations in the Pi calculation have approximately equal workloads, dynamic scheduling provides little benefit. Static and guided scheduling had similar execution times in this experiment.

```
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex4.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex4.c -o lab04_ex4.o
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=1
[./lab04_ex4.o
[Pi = 3.141592653590426
 Time = 0.323759 seconds
[[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=4
 ./lab04_ex4.o
 Pi = 3.141592653589683
 Time = 0.326087 seconds
[[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=8
 ./lab04_ex4.o
 Pi = 3.141592653589815
 Time = 0.324950 seconds
[[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=12
 ./lab04_ex4.o
 Pi = 3.141592653589828
 Time = 0.326370 seconds
[[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=16
 ./lab04_ex4.o
 Pi = 3.141592653589882
 Time = 0.325073 seconds
[[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=20
 ./lab04_ex4.o
 Pi = 3.141592653589850
 Time = 0.324473 seconds
[[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=24
 ./lab04_ex4.o
 Pi = 3.141592653589783
 Time = 0.330159 seconds
[[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex4_shared.c
 [ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex4_shared.c -o lab04_ex4_shared.o
 [ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex4_private.c
[[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex4_private.c -o lab04_ex4_private.o
[[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex4_firstprivate.c
[[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex4_firstprivate.c -o lab04_ex4_firstprivate.o
[[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex4_critical.c
[[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex4_critical.c -o lab04_ex4_critical.o
[[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex4_atomic.c
[[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex4_atomic.c -o lab04_ex4_atomic.o
[[ec2-user@ip-10-0-0-183 ~]$ nano run_ex4.sh
[[ec2-user@ip-10-0-0-183 ~]$ chmod +x run_ex4.sh
[[ec2-user@ip-10-0-0-183 ~]$ ./run_ex4.sh
[==================================
[Threads: 1
[==================================
 Run 1: 0.325750 seconds
 Run 2: 0.325694 seconds
 Run 3: 0.324073 seconds
 Run 4: 0.325066 seconds
 Run 5: 0.324720 seconds
 Run 6: 0.324882 seconds
 Run 7: 0.324088 seconds
 Run 8: 0.325706 seconds
 Run 9: 0.327119 seconds
 Run 10: 0.326257 seconds
 Average (runs 4-10): .325405 seconds
 ==================================
 Threads: 4
 ==================================
```

```
Threads: 4
====================================
Run 1: 0.327286 seconds
Run 2: 0.326814 seconds
Run 3: 0.327172 seconds
Run 4: 0.327850 seconds
Run 5: 0.331442 seconds
Run 6: 0.327896 seconds
Run 7: 0.328042 seconds
Run 8: 0.328448 seconds
Run 9: 0.328203 seconds
Run 10: 0.327832 seconds
Average (runs 4-10): .328530 seconds
====================================
Threads: 8
====================================
Run 1: 0.327340 seconds
Run 2: 0.329211 seconds
Run 3: 0.327021 seconds
Run 4: 0.327235 seconds
Run 5: 0.327555 seconds
Run 6: 0.326688 seconds
Run 7: 0.327017 seconds
Run 8: 0.326976 seconds
Run 9: 0.327080 seconds
Run 10: 0.327358 seconds
Average (runs 4-10): .327129 seconds
====================================
Threads: 12
====================================
Run 1: 0.327429 seconds
Run 2: 0.327232 seconds
Run 3: 0.328031 seconds
Run 4: 0.327866 seconds
Run 5: 0.327724 seconds
Run 6: 0.328402 seconds
Run 7: 0.327217 seconds
Run 8: 0.326907 seconds
Run 9: 0.328683 seconds
Run 10: 0.327287 seconds
Average (runs 4-10): .327726 seconds
====================================
Threads: 16
====================================
Run 1: 0.326627 seconds
Run 2: 0.326578 seconds
Run 3: 0.327880 seconds
Run 4: 0.327907 seconds
Run 5: 0.328053 seconds
Run 6: 0.328556 seconds
Run 7: 0.328308 seconds
Run 8: 0.327824 seconds
Run 9: 0.328241 seconds
Run 10: 0.328216 seconds
Average (runs 4-10): .328157 seconds
====================================
Threads: 20
====================================
Run 1: 0.327715 seconds
Run 2: 0.330151 seconds
```

```
==================================
Threads: 20
==================================
Run 1: 0.327715 seconds
Run 2: 0.330151 seconds
Run 3: 0.327856 seconds
Run 4: 0.327447 seconds
Run 5: 0.328250 seconds
Run 6: 0.423360 seconds
Run 7: 0.330144 seconds
Run 8: 0.328477 seconds
Run 9: 0.328037 seconds
Run 10: 0.327888 seconds
Average (runs 4-10): .341943 seconds
==================================
Threads: 24
==================================
Run 1: 0.328178 seconds
Run 2: 0.327922 seconds
Run 3: 0.329275 seconds
Run 4: 0.329073 seconds
Run 5: 0.327657 seconds
Run 6: 0.329607 seconds
Run 7: 0.328081 seconds
Run 8: 0.327949 seconds
Run 9: 0.328731 seconds
Run 10: 0.327503 seconds
Average (runs 4-10): .328371 seconds
[ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex4_schedule.c
[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex4_schedule.c -o
[ec2-user@ip-10-0-0-183 ~]$ export OMP_NUM_THREADS=24
[[ec2-user@ip-10-0-0-183 ~]$ export OMP_SCHEDULE="static"
[./lab04_ex4_schedule.o
[Pi = 3.141592653589783
Time = 0.327397 seconds
[[ec2-user@ip-10-0-0-183 ~]$ export OMP_SCHEDULE="dynamic"
./lab04_ex4_schedule.o
Pi = 3.141592653589750
Time = 2.679877 seconds
[[ec2-user@ip-10-0-0-183 ~]$ export OMP_SCHEDULE="guided"
./lab04_ex4_schedule.o
Pi = 3.141592653589797
Time = 0.325920 seconds
[[ec2-user@ip-10-0-0-183 ~]$ ./lab04_ex4_shared.o
./lab04_ex4_private.o
./lab04_ex4_firstprivate.o
./lab04_ex4_critical.o
./lab04_ex4_atomic.o
Pi = 3.141592653589782
Time = 0.325304 seconds
[Pi = 3.141592653589783
Time = 0.326629 seconds
Pi = 3.141592653589783
Time = 0.327199 seconds
Pi = 3.141592653589783
Time = 0.327379 seconds
Pi = 3.141592653589783
Time = 0.326689 seconds
```

Exercise 05

Code Segment	Synchronization Method	Sum	Execution Time (seconds)
A	Local sum + Atomic	5,000,050,000	0.001400539
B	Reduction	5,000,050,000	0.000801689
C	Atomic inside loop	5,000,050,000	0.002016679

All three methods produced the correct result of 5,000,050,000. Segment B using reduction was the fastest at 0.000801689 seconds, while Segment C using an atomic operation inside the loop was the slowest at 0.002016679 seconds due to higher synchronization overhead. This shows that the choice and frequency of synchronization can significantly affect parallel performance.

```
 [ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex5_3.c
 [ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex5_1.c
 [ec2-user@ip-10-0-0-183 ~]$ nano lab04_ex5_2.c
[[ec2-user@ip-10-0-0-183 ~]$ gcc -fopenmp lab04_ex5_1.c -o lab04_ex5_1.o
[gcc -fopenmp lab04_ex5_2.c -o lab04_ex5_2.o
[gcc -fopenmp lab04_ex5_3.c -o lab04_ex5_3.o
 [ec2-user@ip-10-0-0-183 ~]$ ./lab04_ex5_1.o
 ./lab04_ex5_2.o
[./lab04_ex5_3.o
 Sum = 5000050000
 Time = 0.001400539 seconds
[Sum = 5000050000
 Time = 0.000801689 seconds
 Sum = 5000050000
 Time = 0.002016679 seconds
```